

Problem 1

For this question, you'll need the following definition:

Definition: A sunlet is a graph with $2n$ vertices that consists of a cycle of length n , and each vertex in the cycle is directly connected to exactly one node of degree one. An example of a sunlet with 12 vertices is shown below.

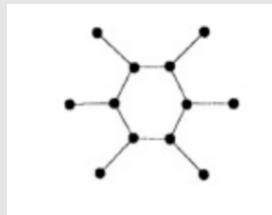


Figure 1: Example of sunlet

Model: Input is an undirected graph G with $2n$ vertices. The algorithm can query an edge (i, j) and it will be told whether or not edge (i, j) is in graph G . Each query has cost 1.

Problem: Output “Yes” if the input graph G is a sunlet, otherwise output “No”.

Your task: Prove that every correct algorithm for the problem has worst-case cost at least $\binom{2n}{2}$

Solution:

Input: We are given an undirected graph of size $2n$.

An algorithm can query whether an edge (i, j) exists or not, and each such query has a cost of 1.

Goal: To prove that any correct algorithm, in the worst case, must make at least $\binom{2n}{2}$ queries to determine whether the graph is a sunlet.

Proof

Observations

1. There are n vertices of degree 2 (forming the cycle).
2. There are n vertices of degree 1 (the leaves).
3. The graph must not have any extra edges beyond the sunlet structure.
4. Any undetected edge could:
 - (i) be an edge forming shortcut in the cycle,
 - (ii) connect leave node to one cycle node,
 - (iii) increase the degree of a node beyond what is allowed.

Adversary Method

- We maintain the possibility of the graph being either a sunlet or not, until all edges are verified.
- No definitive conclusion can be made until every pair of vertices is checked.

Assume we have a algorithm which check fewer than $\binom{2n}{2}$ edges.

Then, there exists at least one edge (a, b) that is not queried.

But, the presence or absence of this edge in the graph can change the result of our decision.

Thus, it is necessary to check all the edges.

Example

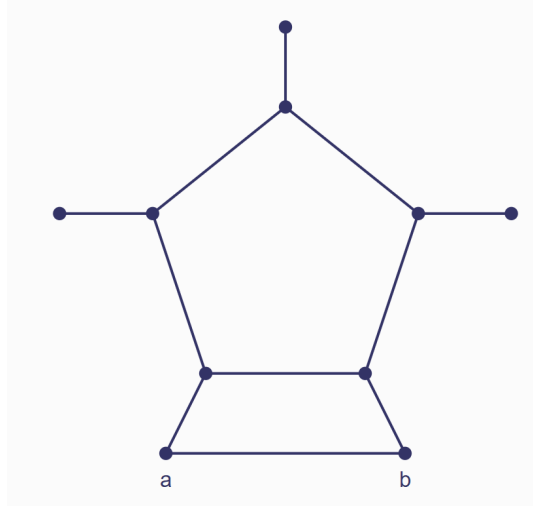


Figure 2: Example illustrating the importance of querying all edges

If we have queried all edges except (a, b) , and based on the rest we decide the graph is a sunlet, then we could be wrong if (a, b) is present and breaks the sunlet structure. Hence, missing even one edge can lead to an incorrect answer.

Conclusion

To correctly determine whether a graph is a sunlet or not, the algorithm must query all possible $\binom{2n}{2}$ pairs of vertices in the worst case.

Worst-case query cost is at least $\binom{2n}{2}$

Problem 2

2. Consider the following model and problem:

Model: For the range of numbers $1, 2, \dots, n$, there is a special threshold value $t \in \{0, 1, \dots, n\}$. For all numbers $i > t$, the number i is considered “unsafe.” All other numbers in the range $1, 2, \dots, n$ are considered “safe.” The algorithm can query any number $i \in \{1, 2, \dots, n\}$, and it will be told whether i is “safe” or “unsafe.” However, if the algorithm ever queries an “unsafe” i , the system shuts down and no further queries are possible.

Problem: Determine the exact value of t .

Your task:

(a) Prove that any algorithm that solves the problem must perform at least n queries in the worst case.

- (b) Let's change the model a bit: suppose that one "unsafe" query is allowed. That is, the system shuts down after exactly two "unsafe" queries. Prove that any algorithm that determines the exact value of t must use $\Omega(\sqrt{n})$ queries in the worst case.
- (c) Design an algorithm that uses only $O(\sqrt{n})$ queries for the problem in part (b).

Solution:

Given: Range of numbers: $1, 2, \dots, n$

t : threshold

Let $i \in [1, n]$, then

if $i > t$, then i is unsafe

if $i \leq t$, then i is safe

We can query any i and determine whether it is safe or unsafe. But if we query an unsafe number, the system shuts down.

Goal: Determine the exact value of t .

(a) Prove: Any algorithm that solves the problem must perform at least n queries in the worst case.

Proof: We cannot randomly perform the queries because if we encounter an unsafe number, the system shuts down, and we will never know the exact value of t .

Thus, to find the exact value of t , we should find the highest value of i such that it is safe.

To do this, the only way we have is to query all the number from 1 to n in increasing order until we find 1st unsafe number.

Now, let's suppose there exists an algorithm which perform less than n query:

In the worst case, if $t = n$, then all elements from 1 to n are safe.

If we perform only $n - 1$ queries and stop, we can't conclude if n is safe or unsafe, so we can't decide if $t = n$ or $n - 1$.

So, to confirm $t = n$, we have to query all n elements to make sure none is unsafe.

Thus, in the worst case, we must perform at least n queries to determine the exact value of t .

(b) Modification: One unsafe query is allowed. On the second unsafe query, the system shuts down.

Claim: Any algorithm that solves the problem must perform at least $\Omega(\sqrt{n})$ queries in the worst case.

Proof:

We must determine the exact value of t .

Case 1: Divide the n numbers into equal-sized blocks.

- Let block size be x .
- Then, the total number of blocks is $\frac{n}{x}$.

- The algorithm queries the last element of each block. After finding the first unsafe number, the system queries all elements of the *previous* block to find the exact value of t .
- The adversary wants to make the job as hard as possible, so it sets $t = n$ (last element is unsafe).
- In the worst case, the total number of queries is:

$$\text{Total} = \frac{n}{x} + x$$

- To minimize the total, differentiate w.r.t. x :

$$\frac{d}{dx} \left(\frac{n}{x} + x \right) = -\frac{n}{x^2} + 1$$

Set derivative to zero:

$$-\frac{n}{x^2} + 1 = 0 \Rightarrow x^2 = n \Rightarrow x = \sqrt{n}$$

Therefore, the minimum number of queries required in the worst case is:

$$\boxed{\Omega(\sqrt{n})}$$

Case II: Exponential Search + Linear Search

We use exponential search to find the first unsafe number or exceed n . The queries are:

$$q_1 = 1, q_2 = 2, q_3 = 4, q_4 = 8, \dots, q_k = 2^{k-1}$$

So the number of exponential queries is approximately k . Then, we have to do linear search in the range:

$$q_{k-1} + 1 \text{ to } q_k - 1 = 2^{k-2} + 1 \text{ to } 2^k - 1$$

The adversary wants to make our job hard, so it will place $t = 2^k = n$. Hence, total queries:

$$q_k = n \Rightarrow 2^{k-1} = n \Rightarrow k = \log_2 n + 1$$

Now, linear search in the last block has size:

$$2^{k-1} - 1 - 2^{k-2} + 1 = 2^{k-2}$$

So, total queries:

$$\text{Total} = \log_2 n + \frac{n}{2}$$

This is not better than $\mathcal{O}(\sqrt{n})$.

Conclusion

Through the analysis above, we have established that any algorithm aiming to determine the exact value of the threshold t must perform at least $\Omega(\sqrt{n})$ queries in the worst case.

Problem (c): Design an algorithm that uses only $\mathcal{O}(\sqrt{n})$ queries for the problem in part (b).

Input:

- n : Range is from 1 to n

Output:

- The exact value of threshold t

Algorithm Idea:

We divide the range into blocks of size $\lceil \sqrt{n} \rceil$, query the last element of each block to find the first unsafe block, then linearly search within that block to find the threshold value t .

Algorithm (Pseudocode):

```
1  Let step =  $\lceil \sqrt{n} \rceil$ 
2  Let k = step
3
4  while (k ≤ n):
5      if isSafe(k):
6          k = k + step
7      else:
8          // We've found the first unsafe block
9          // Search within the previous block
10         for i from (k-step+1) to (k):
11             if not isSafe(i):
12                 return i - 1
13
14 //Handling the Case When  $n$  is Not a Perfect Square
15 for i (k-step+1) to n
16     if not isSafe(i):
17         return i - 1
18 // If all blocks are safe (happens when  $t = n$ )
19 return n
```

Time Complexity:

$$(O(\sqrt{n}))$$